AICO Verification & Quality Assurance

Every automated mechanism that verifies AICO behaves as intended

August 2026

Contents

Verification & quality assurance

This page catalogs every automated mechanism that verifies AICO behaves as intended – from a single line of code to a full statistical fit. It answers a different question from Data inputs, outputs, and measurement points: that page describes what AICO measures about *AI systems*; this one describes what verifies *AICO itself*.

Keep this page current: this is a living reference, traceable to the actual CI configuration (`.github/workflows/*.yml`) and gate scripts (`scripts/`) rather than a description written once and left to drift. Whenever a verification mechanism is added, removed, or its threshold changes, update this page (and regenerate its PDF companion via `scripts/generate_docs_pdfs.py`) in the same change. Counts below are a snapshot as of the date in the PDF footer – re-run the commands in each section for the current figure.

Static analysis

Gate	Tool	Scope	CI job
Lint	`ruff check`	`src/`, `tests/`, `scripts/`, `examples/`, `benchmarks/` (367 Python files)	`lint`
Format	`ruff format --check`	same scope, enforced locally before every commit in this project's own workflow	(part of `lint` conventions; not a separate CI job)
Type check	`mypy --strict`	`src/aico`, `tests`, `scripts`, `benchmarks` (366 Python files)	`typecheck`

Both tools run over every Python file the repository ships or tests with, not a sampled subset. `mypy --strict` additionally enables `warn_unused_ignores` and the Pydantic plugin (`pyproject.toml`, `[tool.mypy]`), so a stale `# type: ignore` or a model field mypy can't see through fails the build the same as a genuine type error.

Security scanning

Gate	Tool	What it checks	CI job
Static security analysis	`bandit -c pyproject.toml -r src/`	Common insecure-code patterns (injection, weak crypto, unsafe deserialization, etc.) across every line of `src/`	`security`
Dependency vulnerabilities	`pip-audit --skip-editable`	Every resolved dependency against the PyPI Advisory Database, on every build	`security`
Software bill of materials	`sbom_json()`	Generates a CycloneDX SBOM (`aico.observatory.security`) from the built wheel's actual dependency graph, uploaded as a build artifact	`build`

Bandit scans the full `src/` tree on every run (37,354 lines of code as of this writing) with one narrowly-scoped, documented suppression (B311, non-cryptographic RNG use – see `[tool.bandit]` in `pyproject.toml`) and reports zero findings otherwise. `pip-audit` is deliberately *not* snapshotted here: CI resolves and audits the exact dependency set fresh on every build, so a newly-published CVE against an already-merged dependency is caught on the next push or PR, not just at merge time – a point-in-time "0 vulnerabilities" claim in this document would go stale within days and should not be trusted over CI's own live result.

Backend test suite

Metric	Count (as of this writing)
Total collected tests	1,855
Fast-tier tests (`pytest -m "not slow"`, used for iteration)	1,788
Slow-tier tests (real NUTS/VI fits, full end-to-end workflows)	67
Frontend unit/component tests (Vitest)	63, across 11 files
Frontend end-to-end tests (Playwright)	131, across 27 files

CI's `test` job runs the **full** suite (fast + slow tiers) with coverage instrumentation; the fast tier alone is what this project's own development workflow runs after every change for quick feedback, with the full suite reserved for pre-merge/CI confirmation given its added runtime (real Bayesian sampling in the slow tier takes minutes, not seconds).

Coverage gate

A dual gate enforced by `scripts/check_coverage_gate.py`, which reads coverage.py's own JSON report and checks line and branch coverage as two *independent* thresholds – a high blended number can otherwise hide a much weaker branch-coverage figure underneath it:

- **Line coverage** ≥ 95%
- **Branch coverage** ≥ 90%

`pyproject.toml`'s `[tool.coverage.report]` additionally sets a blended `fail_under = 95`, enforced by `pytest-cov` itself as a first-pass gate before the more specific script runs. Both gates are evaluated against the **full** suite (fast + slow tiers) in CI, since some code paths – real NUTS/VI sampling, convergence-failure handling – are only exercised by slow tests. A fast-tier-only local run can legitimately sit just under the blended

95% figure while still clearing the line/branch thresholds individually; this is expected, not a regression, and is exactly why CI's gate runs the full suite rather than trusting a fast-tier snapshot.

Scientific validation

Unique to a project whose product is a statistical fit: correctness here means more than "the code runs without error" – it means the *inference itself* recovers known ground truth, produces calibrated uncertainty, and ranks models correctly. `scripts/ run_scientific_validation.py` runs a full simulation study (a synthetic ecosystem forward-sampled from the model's own prior, so ground truth is known exactly) and checks the fitted posterior against it, on a nightly schedule and on any PR touching the model/inference/validation code (`.github/workflows/scientific-validation.yml`).

Gate	Threshold
Parameter-recovery correlation	≥ 0.5
Parameter-recovery RMSE	≤ 1.0
Ranking accuracy (Spearman)	≥ 0.55
95% credible-interval coverage	≥ 0.75
90% credible-interval coverage	≥ 0.6
Scale-stability correlation (Spearman)	≥ 0.85
Forecast 95% coverage	≥ 0.75
Forecast mean absolute error	≤ 0.5

Each run is also compared against a committed baseline (`scripts/scientific_baseline.json`) and fails if any aggregate metric drifts beyond a documented tolerance, catching a slow quality regression that never crosses an absolute floor. A separate CI job (`regression-benchmarks`) runs the validation-focused subset of the ordinary test suite (`-k "validation or baselines or simulation or ablation or experiments or diagnostics"`) on every push for faster feedback between nightly runs.

Frontend verification

Gate	Tool	What it checks
Contract sync	`npm run generate:api` + `git diff --exit-code`	The committed `openapi.d.ts` actually matches the backend's current OpenAPI schema – catches a frontend/backend contract drift before it ships
Types	`tsc --noEmit`	Full TypeScript strict-mode type check
Unit/component tests	Vitest	Pure logic and component-level tests (63 tests)
Production build	`next build`	Also runs ESLint as part of the build; a lint or type error here fails the build
End-to-end + accessibility	Playwright + `@axe-core/playwright`	Every page, in every state a test exercises, is scanned against WCAG 2.0/2.1 A/AA on top of its functional assertions – accessibility is not a separate, optional pass

Build & supply chain

CI's `build` job (gated on every other job passing first) builds the wheel and sdist, installs the wheel into a fresh virtual environment, and imports the package – catching a packaging error (a file missing from the distribution, a broken entry point) that unit tests alone cannot, since they run against the source tree, not the built artifact. The SBOM is generated from that same installed wheel, so it reflects what actually ships, not what `pyproject.toml` merely declares.

Documentation verification

`mkdocs build --strict` (`.github/workflows/docs.yml`) fails the build on any broken internal link or reference across the entire published documentation site, run on every change under `docs/` and deployed to GitHub Pages only after it passes on `main`.

Manual live verification

Automated gates catch what they're written to catch; they do not by themselves confirm a feature works end-to-end in a real deployment. Every phase of this project's own development additionally follows a manual verification discipline before being considered complete: rebuild the Docker images with full build-log inspection (not just an exit code), redeploy, poll container health, confirm the deployed image ID matches what was just built, then exercise the actual feature live via the real HTTP API and, for frontend changes, grep the deployed JavaScript bundle for a string unique to the new code – confirming the running system serves the new build, not a stale cached layer.

`POST /demo/load` (`server.demo_data`) is what makes "exercise the actual feature live" practical without real production data: it forward-samples a full synthetic ecosystem and registers it as real input, so every page has something to render. Its content is deliberately engineered, not filler – three harnesses with distinct cost/accuracy profiles so the Pareto frontier shows a genuine tradeoff, three rating periods so continuous monitoring and the GLMM variance decomposition have something to fit, full model/benchmark coverage so every score-driven page renders – and it is extended alongside every new feature for exactly this reason (the harness maturity and variance-decomposition rows in demo data exist because Phases 4 and 7 added them). `POST /demo/clear` removes exactly what `load` added, leaving any real registered data untouched, so this is safe to run against a shared or already-populated deployment. This is a *functional*-coverage fixture, distinct from the Simulation Lab (`kind=recovery/coverage` above), which validates *statistical* correctness against data with a known ground truth – demo data proves a feature renders and behaves; the Simulation Lab proves the underlying inference is correct.

Summary

Layer	Mechanism	Runs
Style/correctness	ruff, mypy –strict	Every push/PR
Security	bandit, pip-audit, SBOM	Every push/PR (bandit/pip-audit); every build (SBOM)
Backend correctness	1,855 pytest tests, dual coverage gate	Every push/PR
Scientific correctness	8-gate simulation study vs. committed baseline	Nightly + PRs touching model code
Frontend correctness	tsc, Vitest, Playwright + axe, contract sync	Every push/PR
Packaging	Wheel/sdist build, install+import smoke test, SBOM	Every push/PR
Documentation	`mkdocs build --strict`	Every change under `docs/`

Layer	Mechanism	Runs
Deployment reality	Manual Docker rebuild + live curl verification	Every feature phase, before calling it complete

A PDF export of this page is regenerated via `scripts/generate_docs_pdfs.py` whenever this page changes.